CIS 4020 · DATA STRUCTURES

Hashing: Overview and Objectives

Hash functions, collisions, and what a probe really costs

1. What this unit is about

Every searchable structure we have built so far finds an item by comparing it to other items. A sorted array with binary search needs about $\log_2 n$ comparisons. A balanced binary search tree needs about the same. Hashing takes a different approach: instead of searching *for* the item, it *calculates* where the item should be. A hash function $h(x)$ turns the search key x into a table location, and in the best case we go straight there.

That best case is not always what we get. Two keys can land in the same location (a *collision*), and every way of handling collisions costs extra checks, called *probes*. The big idea of this unit is that the number of probes depends mostly on how full the table is, not on how many items it holds. That ratio, the *load factor* α, is what makes hashing close to $O(1)$ when it is kept under control and much worse when it is not.

2. Learning objectives

By the end of this unit, you should be able to do the following.

1. Define search key, hash function, hash value, hash table, collision, probe, load factor, and perfect hash function.
2. Explain why hashing can locate an item in about $O(1)$ time, and why that is an average-case claim, not a guarantee.
3. Describe the qualities of a good hash function: fast to compute, spreads keys uniformly, and uses the entire search key.
4. Compute hash values by hand using selecting digits, folding, and modulus arithmetic, and convert a string key to an integer before hashing it.
5. Insert a sequence of keys into a small table using linear probing, quadratic probing, double hashing, and separate chaining, and show the final table for each.
6. Count the probes needed for a successful search and for an unsuccessful search, and compute the load factor α of a table.
7. Distinguish primary clustering from secondary clustering, and explain which collision strategies suffer from each.
8. Compare hand-counted probe averages to the expected values given by the efficiency formulas, and explain why they do not match exactly.
9. Choose a table size and collision strategy for a given situation, and justify why the table size should be prime and why α should stay below about $2/3$ for open addressing.

3. Key vocabulary

Table 1: Hashing terms used throughout this unit.

Term	Meaning
Search key (x)	The value we use to find an item, such as an ID number or a name.
Hash function ($h(x)$)	The address calculator. Turns a search key into a table index.
Hash table	The array (or array of lists) that stores the items.
Collision	Two different keys produce the same hash value.
Probe	One check of one table location during an insert or a search.
Load factor (α)	Number of items in the table divided by the table size.
Open addressing	Resolving a collision by looking for another open slot in the same array.
Separate chaining	Resolving a collision by keeping a linked list (bucket) at each slot.
Clustering	Occupied slots bunching together, which makes future probes longer.

4. Reading

- **Java (CIS):** Prichard and Carrano, *Data Abstraction and Problem Solving with Java: Walls and Mirrors*, 3rd ed., Chapter 13, "Advanced Implementations of Tables," Section 13.2, "Hashing." Pay special attention to the subsections on resolving collisions and on the efficiency of hashing.
- **C/C++ (CS):** Carrano and Henry, *Secure Data Structures and Algorithms with C++: Walls and Mirrors*, 8th ed., Chapter 24, "Hashing as a Map Implementation." Chapter 23, "Maps and Their Implementations," sets up the ADT that hashing implements.

5. How we will practice

In class we will build a hash table in a spreadsheet: 30 keys, a table of 41 slots, and the function $h(x) = x \bmod 41$, using linear probing, quadratic probing, and double hashing. We will count, by hand, how many probes each search takes, once for keys that are in the table and once for keys that are not. Then we will compare our averages to the efficiency formulas from the textbook. The counts are a rough estimate, not a proof, but they make the formulas feel real: you will see unsuccessful searches cost more than successful ones, and you will see why the table size is a prime number.

6. Where this fits

Hashing is one more way to implement a table (also called a dictionary or a map), alongside the sorted arrays and binary search trees from earlier units. It wins on average-case search, insert, and delete. It loses on anything that needs the items in order, such as printing every item sorted or finding all keys in a range, because a good hash function deliberately scatters the keys. That tradeoff is why both kinds of structures are still in wide use.